LEGACY LOGIC. MODERN ACCESS.

Turn legacy code into reliable, agent-ready APIs and MCP servers.

Connect what runs your business
to what comes next.



THE PROBLEM

Legacy code is trapped. AI agents cannot use it.

01

Core logic is trapped

The rules that run the business live in old systems (COBOL, old Java, Fortran). They will not be rewritten at scale any time soon.

02

Agents cannot reach it

AI agents cannot call that legacy code through MCP or an API.

03

Engineers lose time

Engineers spend their time building connections to old systems instead of inventing and building new things.

UNIQUE VALUE PROPOSITION

Secure, always current, and under your control.

Secure by default

Read-only access. Legacy code runs only in an offline sandbox. Tokens decide who can use what.

Tested before it goes live

Every tool is run against the real legacy code. If the test fails, the tool is not published.

Stays current with the code

When the legacy code changes, a check retests every tool, repairs any that broke, and saves a new version.

Admins stay in control

Only approved employees (admin token) can import systems, repair tools, or roll back to an earlier version.

Every tool links back to its source file and line, and every answer shows which tested version produced it.

THE SOLUTION

Paste a repo.
Publish tested tools.

From legacy source code to agent access, through one checked pipeline.

01

Fetch

Get the code.
Detect the language.

02

Extract

Find the functions.
Link each to its source.

03

Verify

Build it in a sandbox.
Test every function.

04

Publish

Serve it as MCP tools
and a REST API.

Only functions that pass their tests are published.

HOW IT WORKS

From legacy repo to agent-ready API in eight steps

1 fetch

Clone the code at one exact commit.

`sources.py, ingest.py`

2 detect

Find which language or languages it uses.

`ingest.py, runtimes.py`

3 build

Compile or syntax-check it in the Docker sandbox.

`runtime/build.py`

4 extract

List the real functions and programs.

`extract.py`

5 propose

The AI model (Nemotron) drafts tools from that list.

`prompts.py, generate.py`

6 guard

Reject anything that is not real or not safe.

`guard.py`

7 test

Run every tool on the real code and record the results.

`contracts.py`

8 publish

Save a version and serve it over MCP and REST.

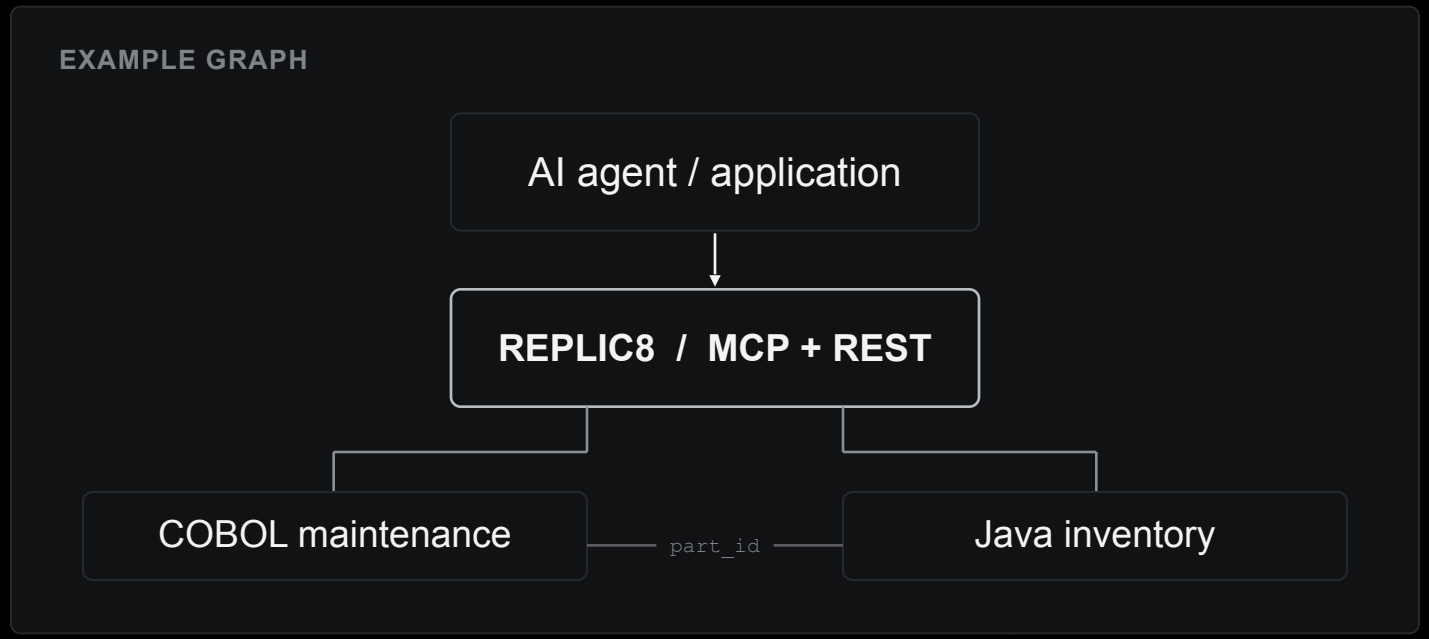
`registry.py, gateway.py`

Only the AI step drafts anything. It never writes code, and every other step is checked code. Output: an MCP server and a REST API.

THE PRODUCT

See all your legacy systems in one graph.

A graph of every legacy repo and its API, showing how they connect and how each one was read.



Every system and API in one view

Each legacy repo shows up as a node, with its tools underneath.

See how systems connect

Shared data fields (like part_id) link one system to another.

See how each tool was read

Each tool shows its source file and line, and how it was parsed.

SECURITY

Your legacy code stays contained and locked down.

READ-ONLY BY DEFAULT

The AI model drafts. REPLIC8 checks.

Write actions are blocked. Nothing is published until it passes its tests.

01 **Offline sandbox**
Legacy code is rebuilt and run only inside a Docker sandbox: no network, read-only files, CPU and memory limits.

02 **Stopped if it runs too long**
Every run has a time limit. Anything that goes over is stopped.

03 **REST API authentication**
Every REST call needs a token, and each token only sees the systems it is allowed to use.

04 **Secure MCP connection**
The MCP endpoint uses the same tokens, checks where requests come from, and only lists the tools a caller may use.

THE CUSTOMER

Built for companies with old systems and AI plans.

Enterprises that inherited or acquired legacy software.

WHO BUYS IT

Platform. Integration. AI engineering.

First use case: give AI agents read-only access to company data.

INDUSTRIES

- Finance
- Insurance
- Energy
- Manufacturing
- Logistics
- Real estate
- Public sector

BUILT FOR WHAT COMES NEXT

Your business logic. Ready for its next life.

Start with one repo and one read-only workflow.



Explore REPLIC8

replic8.work

THE TEAM Jaiom Rola / Aryan / Toni / Krish